

Using a Kernel Density Function and Student's t-distribution to model Value at Risk and Expected Shortfall using Monte Carlo method

Gianni Mazaud
gianni.mazaud@icloud.com

Abstract—Basel III regulation imposes a clear market risk frameworks for financial institutions. The principal shift rests in the core of the attention now turned toward a 97.5% Expected Shortfall, rather than VaR(99%) via FRTB. In this paper we use Monte Carlo simulations through Geometric Brownian motion, Student's t-distribution and non-parametric Kernel Density Estimator (KDE) to calculate VaR and CVaR at the 99% and 97.5% threshold. Empirically, our simulations demonstrate that the Geometric Brownian Motion framework penalizes risk architecture by collapsing the threshold difference between Value at Risk and Expected Shortfall. In stark contrast, both the parametric Student's t-distribution and the non-parametric Kernel Density Estimator successfully express severe market shocks, mapping a distinct, heavy-tailed loss progression deeper into the tail. The data organically captures a severe leptokurtic distribution, validating the Basel III Fundamental Review of the Trading Book (FRTB) guidelines.

Index Terms—Value at Risk, Expected Shortfall, Monte Carlo modeling, Kernel Density Estimator, Student's t-distribution

I. INTRODUCTION

Value at Risk is defined as: "a measure of the worst expected loss on a portfolio of instruments resulting from market movements over a given time horizon and a pre-defined confidence level" [1] [2]. Formally, it can be written as follow:

$$VaR_\alpha = \inf \{l \in \mathbb{R} : \mathbb{P}(L > l) \leq 1 - \alpha\}$$

Expected Shortfall or Conditional Value at Risk is defined as: "a measure of the average of all potential losses exceeding the VaR at a given confidence level" [1]. Mathematically, we can write it as:

$$CVaR_\alpha = \mathbb{E}[L|L \geq VaR_\alpha]$$

Under the Basel III regulation, the confidence level for VaR is $\alpha = 99\%$, even though it focuses primarily on Expected Shortfall. Developed by the Basel Committee on Banking Supervisions (BCBS), this regulation was first implemented in July 2013 and finally reformed in December 2017. Its first aim was to respond to the 2007-2008 global financial crisis. To counter these risks it mainly focuses on enhanced capital requirements, non-risk-based leverage ratio, and counterparty credit risk.

In terms of market risk management, Basel III —specifically through the Fundamental Review of the Trading Book (FRTB)— shifts the core metric from a 99% Value at Risk (VaR) to a 97.5% Expected Shortfall (ES). This shift implies

that regulators no longer satisfied with knowing just the threshold of a bank's potential trading losses, but rather the actual severity of losses when a tail-risk event occurs. Additionally, while the previous VaR framework assumed a static, short-term liquidity horizon across all asset classes, the new CVaR-based framework forces banks to map different liquidity horizons to different risk factors.

Banks aren't talkative on the subject and it's hard to really understand, when they are using the Monte Carlo method, which diffusion model they use [3]. Our goal here will be to compare three different approaches. First, we will use a Geometric Brownian motion, that assumes the logarithm of asset prices follows a Brownian motion with a drift. It implies that the probability of extreme events is highly underestimated. In order to offset that, a Student's t-distribution will be implemented to take into account fat-tail risks. This model is usually a better fit for financial returns and it captures extreme events more accurately. Finally, we are going to use a Kernel Density Estimator which is a non-parametric way to estimate the true distribution of financial returns directly from data.

II. USUAL METHODS

Parametric method

Generally, Basel III allows for three methods: parametric, historic, and Monte Carlo.

Parametric method simply assumes loss follows a Gaussian (normal) distribution, then we have: $L \sim N(\mu_L, \sigma_L^2) \Rightarrow L = \mu_L + \sigma_L U$, with $U \sim N(0, 1)$.

Under this method, the formulas are:

$$VaR_\alpha = \mu_L + \sigma_L \Phi^{-1}(\alpha)$$

Applying this to our definition of Expected Shortfall, it implies:

$$\begin{aligned} CVaR_\alpha &= \mathbb{E}[L|L \geq VaR_\alpha] \\ &= \frac{1}{1 - \alpha} \int_{VaR_\alpha}^{+\infty} l f_L(l) dl \mu_L + \sigma_L \frac{\varphi(\Phi^{-1}(\alpha))}{1 - \alpha} \end{aligned}$$

(See Appendix 1. for demonstrations.)

Historical method

This method rests on the assumption that the worst case scenarios of the period taken into account will remain the same. Nevertheless, reading [3], we can see that the distribution

can still be adapted following some scenarios. Furthermore, it assumes portfolio remains the same, the weight of each asset being unchanged.

Let $r_{i,t}$ be the daily returns for each asset i at date $t \in [i, T]$ our historical interval. We apply today's weight w to each historical return scenario:

$L_t = -\sum_{i=1}^T \omega_i r_{i,t} = -w^T r_t$ (it takes the opposite value since loss is considered the opposite of the asset return). It gives us T realized loss scenarios, using the current portfolio composition replayed through history. We order the T losses in ascending order: $L_{(1)} < L_{(2)} < \dots < L_{(T)}$.

Finally, we have:

$$VaR_\alpha = L_{[\alpha, T]}$$

$$CVaR_\alpha = \frac{1}{T - \alpha T} \sum_{t=\alpha T+1}^T L_{(t)}$$

III. MONTE CARLO METHOD

Method framework

The Monte Carlo method is a way to find answers to hard financial problems by using random numbers and simulating thousands of independent paths following the same distribution. First, we choose a statistical rule for how our assets behave (for instance a log-normal distribution or a Student's t-distribution). It allows us to see where the stock prices can potential go. Finally, we put all these simulated results together from the worst loss to the biggest profit. This gives us a huge list of possibilities, and we just look at the bottom 1% of the worst results to easily find the Value at Risk (VaR) or take the average of those bad results to get the Conditional VaR (CVaR).

Statistical distributions

Our goal now is to implement and compare three different statistical distributions: log-normal, Student's t, and Kernel Density Estimator.

Log-normal distribution

Geometric Brownian motion makes sure prices are strictly positive by using an exponential. Also, it scales volatility and drift based on the current price. Finally, this model assumes that continuously compounded returns are normally distributed. If we assume that an asset S follows a log-normal distribution, we have:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Where S_t is the asset price at time t , μ the drift (expected return), σ the volatility coefficient, dt an infinitesimal time step, dW_t a Wiener process (or Brownian motion) increment, which captures the random market shocks where $dW_T \sim N(0, dt)$ [4]. It has the following properties:

- 1) $W_0 = 0$.

- 2) It has independent increments.

- 3) For any two time steps s and t (where $t > s$), the change in value is normally distributed: $W_t - W_s \sim N(0, t - s)$.

The analytical solution, by applying Itô's Lemma [5] to solve the SDE, gives:

$$S_T = S_0 \exp \left(\left(\mu - \frac{\sigma^2}{2} \right) T + \sigma W_T \right)$$

(Demonstrations are shown in Appendix 2.)

If we set $s = 0$ and look strictly at a single future maturity date $t = T$, the total increment is:

$$W_T - W_0 = W_T \sim N(0, T)$$

If we scale a standard normal random variable $Z \sim N(0, 1)$ by multiplying it by $\sqrt{T}$, its mean remains 0 and its variance becomes $(\sqrt{T})^2 = T$. Therefore:

$$W_T \stackrel{d}{=} Z\sqrt{T}$$

Since W_T can be written as $Z\sqrt{T}$, where $Z \sim N(0, 1)$, the formula used is:

$$S_T = S_0 \exp \left(\left(\mu - \frac{\sigma^2}{2} \right) T + \sigma Z\sqrt{T} \right)$$

Student's T-distribution [6]

The Student's t-distribution does not only rely on the mean and variance, but introduces a third parameter: degrees of freedom (ν). Further in this paper, we use $\nu = 4$ arbitrarily because it keeps our core risk equations from breaking (finite variance) while aggressively preparing our VaR model for severe market crashes. The mathematical formula for the Probability Density Function (PDF) of such a distribution is:

$$f(x) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

Where the Gamma function is defined as:

$$\Gamma(z) = \int_0^{+\infty} t^{z-1} e^{-t} dt$$

If the degrees of freedom is low ν the distribution has very fat tails and a higher peak (leptokurtic). However when $\nu \rightarrow \infty$ the distribution mathematically converges into the standard normal distribution.

Modeling an asset using this distribution, we get:

$$S_T = S_0 \exp \left(\mu T + \sigma \sqrt{\frac{\nu-2}{\nu}} T_\nu \sqrt{T} \right)$$

Where T_ν is a random variable drawn from a standard Student's t-distribution with ν degrees of freedom and $\sqrt{\frac{\nu-2}{\nu}}$ is the scaling factor ensuring that the variance of the return over time matches our target volatility. (Proof is given in Appendix 3.)

Kernel Density Estimator

The formal mathematical formula for a Kernel Density Estimator is:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

Where h is the bandwidth, and $K(\cdot)$ the Kernel function. If we use the Gaussian (Normal) Kernel: $K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2}$, we finally get:

$$\hat{f}_h(x) = \frac{1}{nh\sqrt{2\pi}} \sum_{i=1}^n e^{-\frac{1}{2}\left(\frac{x - x_i}{h}\right)^2}$$

the summation gives us exactly the probability density function (PDF) of a normal distribution $N(x_i, h^2)$. Therefore, the KDE can be written as:

$$\hat{f}_h(x) = \frac{1}{n} \sum_{i=1}^n f_N(x|\mu = x_i, \sigma^2 = h^2)$$

This tells us that the KDE treats the asset return distribution as a uniform mixture of n distinct normal distributions. Assuming that $y_j \sim N(x_i, h^2)$ we have that:

$$y_j = x_i + hZ_j$$

To decide which value to choose for h we will use the Silverman's Rule of Thumb [7], that states:

$$h = \left(\frac{4\hat{\sigma}^5}{3n}\right)^{\frac{1}{5}} \approx 1.05\hat{\sigma}n^{-\frac{1}{5}}$$

The core difference of this approach compared to a Brownian motion is that it assume no prior functional form for the distribution. Instead of forcing the data into a bell curve, the data defines its own shape.

How to decide which distribution model to use?

KDE (Non-Parametric) does not assume any underlying shape for our data. It builds a distribution directly from historical data points by placing a "kernel" (smooth curve) over each observation. It is purely empirical. Student's t (Parametric) assumes a fixed mathematical shape defined by specific parameters (mean, scale, and degrees of freedom). It is symmetrical but features heavy tails, making it excellent for capturing extreme financial shocks. GBM (Stochastic Process) unlike the other two, GBM is not a static probability distribution—it is a time-dependent path generator. It models how an asset price moves over time based on a constant drift (trend) and volatility (random walk).

IV. IMPLEMENTATIONS & RESULTS

Using Python, we can simulate an asset N times and adjust the characterizing parameters as required for our study. The python code can be found in Appendix 4. For our study, we construct an equally-weighted portfolio made of: S&P 500, NASDAQ, CAC40, FTSE 100, DAX, and Hang Seng. We extract the data for a five years period. Mean and standard deviation are computed via a Maximum Likelihood estimation which provides a uniform, consistent framework to compare

the normal distribution against non-Gaussian models (like the Student's t -distribution) that require statistical fitting rather than simple averages. After gaining these data, we model 50,000 paths for each distribution. To get a better picture of how returns are distributed, here are the histograms representing the distribution of our generated-data.

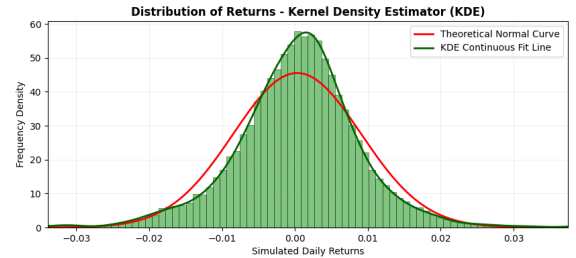
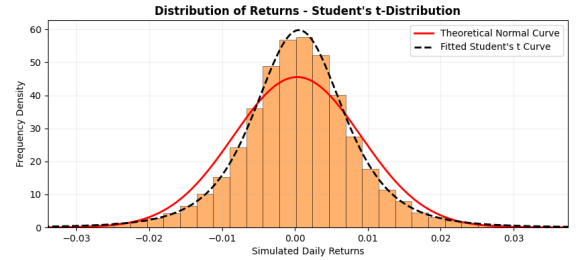
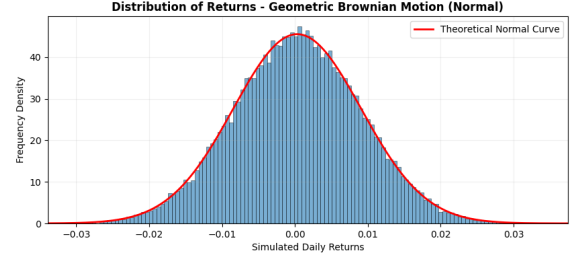


TABLE I: Statistical Significance Tests for Skewness and Kurtosis

Model	Test / Metric	Value / Stat	z-score	p-value
GBM	Skewness	-0.0095	-0.8703	0.384119
	Ex. Kurtosis	+0.0010	+0.0465	0.962946
	D'Agostino K^2	+0.7616	—	0.683322
Student's t	Skewness	+0.0004	+0.0330	0.973636
	Ex. Kurtosis	+14.7932	+675.2208	0.000000***
	D'Agostino K^2	+10632.5134	—	0.000000***
KDE	Skewness	-1.0377	-94.7309	0.000000***
	Ex. Kurtosis	+11.2080	+511.5783	0.000000***
	D'Agostino K^2	+15497.5323	—	0.000000***

Significance codes: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

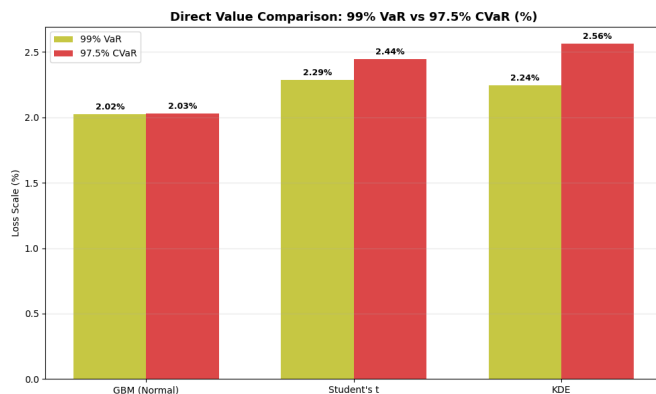
The Geometric Brownian Motion (GBM) model assumes that asset returns follow a perfect normal distribution. In our

simulation, the GBM skewness yields a z-score of -0.8703 ($p = 0.384$), and the excess kurtosis yields a z-score of $+0.0465$ ($p = 0.963$). Because both p-values are much larger than 0.05, these results are not statistically significant, meaning the distribution is perfectly Gaussian. Consequently, the GBM model is completely blind to unexpected market shocks or severe tail risks, making it an unrealistic tool for downside risk management.

To fix the limitations of the normal model, a Student's t-distribution was fitted to the data. While the skewness remains statistically insignificant ($z = +0.0330$, $p = 0.974$), the excess kurtosis shifts drastically, showing an extreme z-score of $+675.2208$ ($p < 0.001$). This highly significant p-value proves that the asset's returns have much fatter tails than a normal distribution. On the chart, the Student's t curve successfully captures the heavy risk in the tails, making it a much more reliable model for calculating portfolio Value-at-Risk (VaR).

Although the Student's t-distribution models fat tails well, its mathematical structure forces the data to be perfectly symmetric. The non-parametric Kernel Density Estimator (KDE) removes this restriction and reveals a highly significant negative skewness with a z-score of -94.7309 ($p < 0.001$), alongside a significant excess kurtosis z-score of $+511.5783$ ($p < 0.001$). These significant p-values confirm that the distribution is heavily distorted to the left. This proves the true asymmetry of financial risk: asset returns typically face sudden, severe negative crashes rather than positive jumps.

Finally, putting altogether our estimated risk measures:

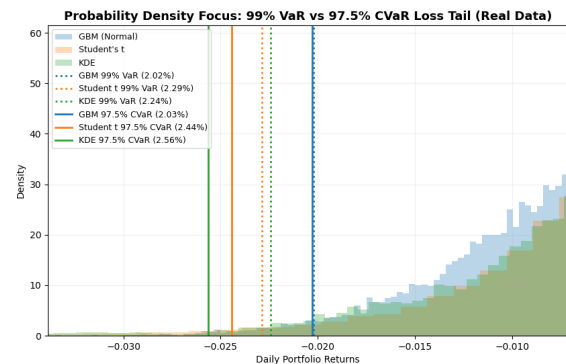


This histogram shows a big difference between the models. Under the normal GBM model, the $VaR(99\%) = 2.02\%$ and the $CVaR(97.5\%) = 2.03\%$ are almost exactly the same, with a gap of only 0.01%. This happens because the normal model assumes that extreme losses are nearly impossible. In the real world, using this normal model is dangerous because it leaves a portfolio unprotected against big market drops.

When we look at the other models, the risk numbers increase to show the true dangers of the market. The Student's t-distribution gives a $VaR(99\%)$ of 2.29% and a $CVaR(97.5\%)$ of 2.44%, showing a larger gap of 0.15%

because it captures fat tails. However, the KDE model gives the most realistic view of risk. Even though its $VaR(99\%)$ is 2.24%, its $CVaR(97.5\%)$ jumps to the highest level at 2.56%. This 0.32% gap is caused by the negative skewness of the data, proving that real-world market crashes are much worse than simple models predict.

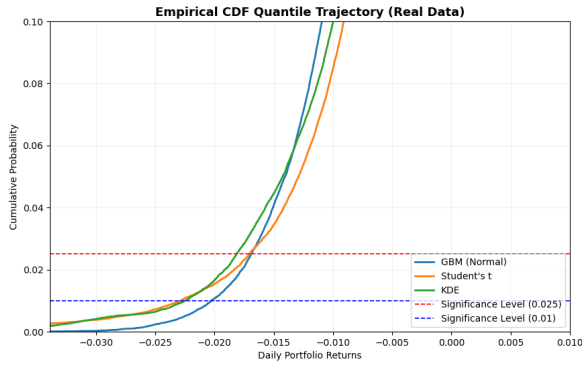
Also, we notice that in every case $CVaR(97.5\%)$ is above $VaR(99\%)$ reflecting the reinforcement in capital exposition regulation imposed by Basel III.



The graph of Probability Density shows the worst-case losses for the three different models. The normal model (GBM) is represented by the blue lines. Here, the $VaR(99\%)$ is 2.02% and the $CVaR(97.5\%)$ is 2.03%. Because these two lines are almost on top of each other, the model assumes that there are no risk of extreme scenarios.

The other two models look at the actual danger in the market much better. As seen in the graph, the Student's t distribution (orange lines) separates the VaR (2.29%) and CVaR (2.44%) because it takes into consideration the market has fat tails. However, the KDE model (green lines) shows the most realistic danger. Even though its $VaR(99\%)$ is 2.24%, its $CVaR(97.5\%)$ moves way to the left at 2.56%. This large gap happens because real financial data has a negative skew, meaning that when markets crash, they fall fast and deep.

$CVaR(97.5\%)$ is always represented at the left (deeper into high-loss territory). Unlike the normal distribution, KDE and Student's t-histogram bars continue to exist on the left part of the graph. This illustrates again the reason why Basel III guidelines transitioned regulatory capital calculations to a $CVaR(97.5\%)$ framework.



The chart empirical CDF shows the cumulative probability of negative return. The two horizontal dashed lines represent the risk thresholds: the red line is the 2.5% risk level (used for 97.5% CVaR), and the blue line is the 1% risk level (used for 99% VaR). This chart shows exactly where each model intersects these risk lines.

As we look more closely, we can see how the lines cross the thresholds differently.

- The blue GBM (blue) line stays to the right of the others.
- However, both the Student's t (orange) and KDE (green) lines are much further to the left.

The fact that the orange and green curves sit higher up on the left side proves that large losses happen much more often in real life than the normal GBM model expects.

V. CONCLUSION

This paper introduces three distributions to evaluate Value at Risk and Expected Shortfall under the Basel III (FRTB) regulation. We modeled 50,000 different random paths for each distribution in order to evaluate these risk measures.

The first distribution used is a geometric Brownian motion (using Gaussian distribution assumptions), and our findings show that it fails to capture fat-tail risk. Indeed, as calculated, the excess kurtosis is near zero showing the lack of consideration for extreme scenarios.

The second distribution implemented is the Student's t-distribution in order to overcome that issue of not considering fat-tail risk. This distribution exhibits a clearly positive excess kurtosis which implies a leptokurtic distribution as we can observe with the corresponding histogram. Finally, the Kernel Density Estimator also exhibits a leptokurtic distribution and a clearly lower skewness than the two others. As the data tells, this distribution gives the highest Value at Risk but still has a lower Expected Shortfall, relative to the thresholds imposed by the regulation. This model is different in the fact that it is non-parametric, it let the data defines it own form.

Our findings demonstrate the danger of normality assumption. As we showed, the Gaussian distribution does not take consideration of fat-tail risks that are essential in risk measurement. On the one side, data clearly shows the geometric Brownian motion exhibits lower values for both measures (Expected

Shortfall and Value at Risk) and gives almost the value for both of them showing again the ignorance of extreme scenarios. On the other side, KDE as well as Student's t distributions have a much higher CVaR(97.5%) than VaR(99%) proving the utility of the Basel III regulation that shifts toward the first one in order to reinforce capital requirement.

A parametric approach assumes that the distribution of a portfolio returns follows a known statistical distribution equation. To capture fat tails with a parametric model, Gaussian distribution must be forgotten. In a non-parametric framework like the Kernel Density Estimator (KDE), we get rid of equations to use data directly. Then, it captures leptokurtosis naturally. This gives more accurate measures, especially for hidden structural risks (such as the negative skewness caused by market sell-offs).

REFERENCES

- [1] Basel Committee on Banking Supervision, "Explanatory note on the minimum capital requirements for market risk," 2019. [Online]. Available: https://www.bis.org/bcbs/publ/d457_note.pdf
- [2] J.-P. Laurent, "Modèle structurel du risque de crédit," http://laurent.jeanpaul.free.fr/Enseignement/GRF_2023-24_modele_structurel_du_risque_de_credit.pdf, 2023, accessed: 2026-06-04.
- [3] Société Générale Group, "2025 universal registration document: 2024 annual financial report," Société Générale, Universal Registration Document Form 297, March 2025, filed with the Autorité des Marchés Financiers (AMF) on March 12, 2025. [Online]. Available: <https://usprogram.socgen.com/files/297.pdf>
- [4] M. Csörgő, "Brownian motion-wiener process," *Canadian Mathematical Bulletin*, vol. 22, no. 3, pp. 257–279, 1979. [Online]. Available: <https://www.cambridge.org/core/journals/canadian-mathematical-bulletin/article/brownian-motionwiener-process/17DB3F7FE5C4F6216E6A214D166D44A6>
- [5] K. Itô, "On stochastic differential equations," *Memoirs of the American Mathematical Society*, vol. 4, pp. 1–51, 1951, cited by: 3105. [Online]. Available: <https://doi.org/10.1090/memo/0004>
- [6] Student, "The probable error of a mean," *Biometrika*, vol. 6, no. 1, pp. 1–25, 1908.
- [7] B. W. Silverman, *Density Estimation for Statistics and Data Analysis*, ser. Monographs on Statistics and Applied Probability. London: Chapman and Hall, 1986, vol. 26.

VI. APPENDICES

Appendix 1. — demonstrations for parametric method
VaR is such that [2]:

$$\mathbb{P}(L < VaR_\alpha) = \alpha$$

Since we assumed $L \sim N(\mu_L, \sigma_L)$:

$$\Leftrightarrow \mathbb{P}(\mu_L + \sigma_L U < VaR_\alpha) = \alpha$$

With $U \sim N(0, 1)$, We can simplify as:

$$\Leftrightarrow \mathbb{P}\left(U < \frac{VaR_\alpha - \mu_L}{\sigma_L}\right) = \alpha$$

$$\Leftrightarrow \Phi\left(\frac{VaR_\alpha - \mu_L}{\sigma_L}\right) = \alpha$$

$$\Leftrightarrow \frac{VaR_\alpha - \mu_L}{\sigma_L} = \Phi^{-1}(\alpha) \Leftrightarrow VaR_\alpha = \mu_L + \sigma_L \Phi^{-1}(\alpha)$$

Applying this to our definition of Expected Shortfall, it implies:

$$CVaR_\alpha = \mathbb{E}[L|L \geq VaR_\alpha] = \frac{1}{1-\alpha} \int_{VaR_\alpha}^{+\infty} l f_L(l) dl$$

with f_L the probability density function of $L \sim N(\mu_L, \sigma_L^2)$.
Hence:

$$f_L(l) = \frac{1}{\sqrt{2\pi\sigma_L^2}} \exp\left(-\frac{(l-\mu_L)^2}{2\sigma_L^2}\right)$$

Let

$$\begin{aligned} \varphi : \frac{l-\mu_L}{\sigma_L} &\rightarrow \frac{1}{\sqrt{2\pi}} \exp\left(-\left(\frac{l-\mu_L}{\sigma_L}\right)^2 / 2\right) \\ &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(l-\mu_L)^2}{2\sigma_L^2}\right) \end{aligned}$$

This gives us:

$$f_L(l) = \frac{1}{\sigma_L} \varphi\left(\frac{l-\mu_L}{\sigma_L}\right)$$

Finally, we can compute the Expected Shortfall:

$$CVaR_\alpha(L) = \frac{1}{1-\alpha} \int_{VaR_\alpha(L)}^{+\infty} \frac{l}{\sigma_L} \varphi\left(\frac{l-\mu_L}{\sigma_L}\right) dl$$

Let's define:

$$z = \frac{l-\mu_L}{\sigma_L} \Leftrightarrow l = \mu_L + \sigma_L z \Rightarrow dl = \sigma_L dz$$

This gives us:

$$\frac{1}{\sigma_L} \varphi\left(\frac{l-\mu_L}{\sigma_L}\right) \sigma_L dz = \varphi(z) dz$$

$$VaR_\alpha = \mu_L + \sigma_L \Phi^{-1}(\alpha)$$

$$z = \frac{l-\mu_L}{\sigma_L} = \frac{(\mu_L + \sigma_L \Phi^{-1}(\alpha) - \mu_L)}{\sigma_L} = \Phi^{-1}(\alpha)$$

Finally:

$$CVaR_\alpha(L) = \frac{1}{1-\alpha} \int_{\Phi^{-1}(\alpha)}^{+\infty} (\mu_L + \sigma_L z) \varphi(z) dz$$

$$CVaR_\alpha = \frac{1}{1-\alpha} \left(\mu_L \int_{\Phi^{-1}(\alpha)}^{+\infty} \varphi(z) dz + \sigma_L \int_{\Phi^{-1}(\alpha)}^{+\infty} z \varphi(z) dz \right)$$

Let's compute separately these two integrals:

$$I_1 = \int_{\Phi^{-1}(\alpha)}^{+\infty} \varphi(z) dz = 1 - \Phi(\Phi^{-1}(\alpha)) = 1 - \alpha$$

$$I_2 = \int_{\Phi^{-1}(\alpha)}^{+\infty} z \varphi(z) dz$$

$$= - \int_{\Phi^{-1}(\alpha)}^{+\infty} \varphi'(z) dz = - \left[\varphi(z) \right]_{\Phi^{-1}(\alpha)}^{+\infty} = \varphi(\Phi^{-1}(\alpha))$$

We use here two arguments:

- $\varphi'(z) = -z\varphi(z)$
- $\varphi(z) \rightarrow 0$ when $z \rightarrow +\infty$

Hence:

$$CVaR_\alpha = \frac{1}{1-\alpha} [\mu_L(1-\alpha) + \sigma_L \varphi(\Phi^{-1}(\alpha))]$$

$$\Leftrightarrow CVaR_\alpha = \mu_L + \sigma_L \frac{\varphi(\Phi^{-1}(\alpha))}{1-\alpha}$$

Appendix 2. — Brownian motion

We let the following formula:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Let: $f(S_t) = \ln(S_t)$. For a continuous function $f(t, S_t)$ the second-order Taylor expansion using Itô's Lemma [5] gives us:

$$df = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S} dS_t + \frac{1}{2} \frac{\partial^2 f}{\partial S^2} (dS_t)^2$$

Let's calculate the partial derivatives for our specific function f :

$$\frac{\partial f}{\partial t} = 0$$

$$\frac{\partial f}{\partial S} = \frac{1}{S_t}$$

$$\frac{\partial^2 f}{\partial S^2} = -\frac{1}{S_t^2}$$

Substituting these derivatives back into Itô's formula yields:

$$d(\ln(S_t)) = \frac{1}{S_t} dS_t + \frac{1}{2} \left(-\frac{1}{S_t^2} (dS_t)^2 \right)$$

To evaluate the squared differential term $(dS_t)^2$, let's square our original SDE expression:

$$(dS_t)^2 = (\mu S_t dt + \sigma S_t dW_t)^2$$

$$(dS_t)^2 = \mu^2 S_t^2 (dt)^2 + 2\mu\sigma S_t^2 dt dW_t + \sigma^2 S_t^2 (dW_t)^2$$

Knowing the following properties:

- $dt \cdot dt \rightarrow 0$
- $dt \cdot dW_t \rightarrow 0$
- $dW_t \cdot dW_t \rightarrow dt$

We finally get:

$$(dS_t)^2 = \sigma^2 S_t^2 dt$$

By substituting our original expression for dS_t and our newly calculated $(dS_t)^2$ into the equation, we have:

$$d(\ln(S_t)) = \frac{1}{S_t} (\mu S_t dt + \sigma S_t dW_t) - \frac{1}{2 S_t^2} (\sigma^2 S_t^2 dt)$$

We distribute the terms across the fractions:

$$d(\ln(S_t)) = \mu dt + \sigma dW_t - \frac{1}{2} \sigma^2 dt$$

$$\Leftrightarrow d(\ln(S_t)) = \left(\mu - \frac{\sigma^2}{2}\right) dt + \sigma dW_t$$

We can directly integrate both sides from the initial time ($t = 0$) to our final maturity horizon ($t = T$):

$$\int_0^T d(\ln S_t) = \int_0^T \left(\mu - \frac{\sigma^2}{2}\right) dt + \int_0^T \sigma dW_t$$

$$\Leftrightarrow \ln(S_T) - \ln(S_0) = \left(\mu - \frac{\sigma^2}{2}\right) T + \sigma(W_T - W_0)$$

$$\Leftrightarrow \ln\left(\frac{S_T}{S_0}\right) = \left(\mu - \frac{\sigma^2}{2}\right) T + \sigma W_T$$

$$\Leftrightarrow S_T = S_0 \exp\left(\left(\mu - \frac{\sigma^2}{2}\right) T + \sigma W_T\right)$$

Appendix 3. — Student's t-distribution

We want to show that:

$$\lim_{\nu \rightarrow \infty} f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

. The PDF is given by:

$$f(x) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

If we analyze this into two independent parts, and using the fundamental definition of the exponential constant $\lim_{n \rightarrow +\infty} (1 + \frac{a}{n})^n = e^a$, we evaluate the limit of the right-hand term:

$$\lim_{\nu \rightarrow \infty} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}} = \lim_{\nu \rightarrow \infty} \left[\left(1 + \frac{x^2}{\nu}\right)^\nu\right]^{-\frac{1}{2}} \cdot \left(1 + \frac{x^2}{\nu}\right)^{-\frac{1}{2}}$$

Thus, the variable part yields exactly the Gaussian kernel: $e^{-\frac{x^2}{2}}$

Using the Stirling's approximation for the Gamma function, which states that for large z :

$$\Gamma(z) \sim \sqrt{\frac{2\pi}{z}} \left(\frac{z}{e}\right)^z$$

Let's apply this to the ratio $\frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})}$ by setting $z = \frac{\nu}{2}$:

$$\begin{aligned} \frac{\Gamma(z + \frac{1}{2})}{\Gamma(z)} &\sim \frac{\sqrt{\frac{2\pi}{z+1/2}} \left(\frac{z+1/2}{e}\right)^{z+1/2}}{\sqrt{\frac{2\pi}{z}} \left(\frac{z}{e}\right)^z} \\ &= \sqrt{\frac{z}{z+1/2}} \cdot e^{-1/2} \cdot \frac{(z+1/2)^{z+1/2}}{z^z} \end{aligned}$$

Rearranging the last fraction:

$$\frac{(z+1/2)^z \cdot (z+1/2)^{1/2}}{z^z} = \left(1 + \frac{1}{2z}\right)^z \cdot \sqrt{z+1/2}$$

Taking the limit as $z \rightarrow \infty$:

- $\sqrt{\frac{z}{z+1/2}} \rightarrow 1$
- $\left(1 + \frac{1}{2z}\right)^z \rightarrow e^{1/2}$

Hence:

$$\lim_{z \rightarrow \infty} \frac{\Gamma(z + \frac{1}{2})}{\Gamma(z)} = \sqrt{z + \frac{1}{2}} \sim \sqrt{z} = \sqrt{\frac{\nu}{2}}$$

If we substitute this back into the normalization constant multiplier:

$$\lim_{\nu \rightarrow \infty} \frac{1}{\sqrt{\nu\pi}} \frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})} = \frac{1}{\sqrt{\nu\pi}} \cdot \sqrt{\frac{\nu}{2}} = \frac{1}{\sqrt{2\pi}}$$

Combining both parts gives the standard normal density function:

$$f(x) \rightarrow \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

Let T_ν be a standard Student's t-distributed random variable with ν degrees of freedom. By definition: $\mathbb{E}[T_\nu] = 0$ and $\mathbb{V}(T_\nu) = \frac{\nu}{\nu-2}$ for $\nu > 2$.

To scale a random variable so that its variance equals 1, we must divide by its standard deviation:

$$Z = \frac{T_\nu}{\sqrt{\mathbb{V}(T_\nu)}} = \frac{T_\nu}{\sqrt{\frac{\nu}{\nu-2}}} = T_\nu \sqrt{\frac{\nu-2}{\nu}}$$

Now Z represents a zero-mean, unit-variance process containing the heavy tails dictated by ν .

Under continuous geometric scaling over a time horizon T , with an expected annualized return μ and annualized volatility σ , the total return volatility scales by $\sqrt{T}$. We inject Z into the return dynamics:

$$\ln\left(\frac{S_T}{S_0}\right) = \mu T + \sigma \sqrt{T} Z$$

$$\Leftrightarrow \ln\left(\frac{S_T}{S_0}\right) = \mu T + \sigma \sqrt{T} \left(T_\nu \sqrt{\frac{\nu-2}{\nu}}\right)$$

$$\Leftrightarrow \ln\left(\frac{S_T}{S_0}\right) = \mu T + \sigma \sqrt{\frac{\nu-2}{\nu}} T_\nu \sqrt{T}$$

$$\Leftrightarrow S_T = S_0 \exp\left(\mu T + \sigma \sqrt{\frac{\nu-2}{\nu}} T_\nu \sqrt{T}\right)$$

Appendix 4. — Python code

```
1 import numpy as np
2 import scipy.stats as stats
3 import matplotlib.pyplot as plt
4
5 # Set global seed for exact reproducibility across all
6 # random generations
7 np.random.seed(42)
8
9 # =====
10 # 1. USER-DEFINED INPUT PARAMETERS
11 # =====
12 target_drift = -0.001 # mu (e.g., -0.1% daily
13 # expected return)
14 target_volatility = 0.015 # sigma (e.g., 1.5% daily
15 # volatility)
```

```

13 target_df = 4                # Degrees of freedom (controls
    ↳ historical fat-tail weight)
14
15 n_days = 1000                # Number of "historical" days
    ↳ to generate
16 M = 50000                    # Number of Monte Carlo
    ↳ scenarios
17 alpha = 0.975                # Confidence level for baseline
    ↳ CDF plotting (Plot 2)
18
19 initial_price = 100           # Starting asset price for
    ↳ trajectory paths simulation
20 n_path_steps = 100           # Horizon length (in days) to
    ↳ project the sample paths
21 n_paths_to_plot = 15         # Number of random paths to
    ↳ visualize per method
22
23 print("INITIALIZING PORTFOLIO STRESS TESTING ENGINE...")
24 print(f"Configured Daily Drift: {target_drift:.4%}")
25 print(f"Configured Daily Volatility:
    ↳ {target_volatility:.4%}\n" + "-"*60)
26
27 # =====
28 # 2. GENERATE THE HISTORICAL BASELINE
29 # =====
30 # Simulate random values from a Student's t-distribution
    ↳ scaled to user specifications
31 historical_returns = (np.random.standard_t(df=target_df,
    ↳ size=n_days) * target_volatility) + target_drift
32
33
34 # =====
35 # APPROACH 1: GEOMETRIC BROWNIAN MOTION (NORMAL)
36 # =====
37 # GBM extracts parameters via Maximum Likelihood
    ↳ Estimation, assuming perfect normality
38 mu_norm, sigma_norm = stats.norm.fit(historical_returns)
39
40 # 1A. Returns for VaR/CVaR static horizon
41 sim_returns_gbm = np.random.normal(loc=mu_norm,
    ↳ scale=sigma_norm, size=M)
42
43 # 1B. Dynamic path generation matrix (steps x paths)
44 paths_returns_gbm = np.random.normal(loc=mu_norm,
    ↳ scale=sigma_norm, size=(n_path_steps,
    ↳ n_paths_to_plot))
45 price_paths_gbm = initial_price *
    ↳ np.exp(np.vstack([np.zeros(n_paths_to_plot),
    ↳ np.cumsum(paths_returns_gbm, axis=0)]))
46
47
48 # =====
49 # APPROACH 2: STUDENT'S T-DISTRIBUTION
50 # =====
51 # Fits parameters directly to map history onto a
    ↳ theoretical t-curve
52 df_est, loc_est, scale_est =
    ↳ stats.t.fit(historical_returns)
53
54 # 2A. Returns for VaR/CVaR static horizon
55 sim_returns_t = stats.t.rvs(df=df_est, loc=loc_est,
    ↳ scale=scale_est, size=M)
56
57 # 2B. Dynamic path generation matrix
58 paths_returns_t = stats.t.rvs(df=df_est, loc=loc_est,
    ↳ scale=scale_est, size=(n_path_steps, n_paths_to_plot))
59 price_paths_t = initial_price *
    ↳ np.exp(np.vstack([np.zeros(n_paths_to_plot),
    ↳ np.cumsum(paths_returns_t, axis=0)]))
60
61
62 # =====
63 # APPROACH 3: KERNEL DENSITY ESTIMATION (KDE)
64 # =====
65 # Fits a non-parametric Gaussian KDE over the historical
    ↳ sample returns
66 kde = stats.gaussian_kde(historical_returns,
    ↳ bw_method='scott')
67 h = kde.factor * np.std(historical_returns)
68
69 # 3A. Returns for VaR/CVaR static horizon (Hierarchical
    ↳ sampling)
70 random_indices = np.random.choice(n_days, size=M,
    ↳ replace=True)
71
72 x_I = historical_returns[random_indices]
73 Z = np.random.normal(0, 1, M)
74 sim_returns_kde = x_I + h * Z
75
76 # 3B. Dynamic path generation matrix via structured KDE
    ↳ drawing loops
77 paths_returns_kde = np.zeros((n_path_steps,
    ↳ n_paths_to_plot))
78 for step in range(n_path_steps):
79     idx = np.random.choice(n_days, size=n_paths_to_plot,
    ↳ replace=True)
80     paths_returns_kde[step, :] = historical_returns[idx] +
    ↳ h * np.random.normal(0, 1, n_paths_to_plot)
81 price_paths_kde = initial_price *
    ↳ np.exp(np.vstack([np.zeros(n_paths_to_plot),
    ↳ np.cumsum(paths_returns_kde, axis=0)]))
82
83 # =====
84 # RISK METRICS EVALUATION FUNCTION
85 # =====
86 def calculate_var_cvar(sim_returns, alpha=0.975):
87     """
88     Sorts returns and extracts Value at Risk and
    ↳ Conditional Value at Risk.
89     Returns are transformed to positive numbers
    ↳ representing losses.
90     """
91     sorted_returns = np.sort(sim_returns)
92     k = int(np.floor((1 - alpha) * len(sorted_returns)))
93
94     var = -sorted_returns[k]
95     cvar = -np.mean(sorted_returns[:k])
96     return var, cvar
97
98 # =====
99 # COMPUTE SYNCHRONIZED TARGET METRICS
100 # =====
101 # Calculate specific cross-quantile targets for Plot 1 and
    ↳ Plot 3
102 var_gbm_99, _ = calculate_var_cvar(sim_returns_gbm,
    ↳ alpha=0.99)
103 var_t_99, _ = calculate_var_cvar(sim_returns_t,
    ↳ alpha=0.99)
104 var_kde_99, _ = calculate_var_cvar(sim_returns_kde,
    ↳ alpha=0.99)
105
106 _, cvar_gbm_975 = calculate_var_cvar(sim_returns_gbm,
    ↳ alpha=0.975)
107 _, cvar_t_975 = calculate_var_cvar(sim_returns_t,
    ↳ alpha=0.975)
108 _, cvar_kde_975 = calculate_var_cvar(sim_returns_kde,
    ↳ alpha=0.975)
109
110 # =====
111 # RESULTS PRINT OUT
112 # =====
113 print(f"{'Approach':<30} | {'99.0% VaR':<10} | {'97.5%
    ↳ CVaR':<10}")
114 print("-" * 60)
115 print(f"{'1. Geometric Brownian (Normal)':<30} |
    ↳ {var_gbm_99:.4%} | {cvar_gbm_975:.4%}")
116 print(f"{'2. Student's t-Distribution':<30} |
    ↳ {var_t_99:.4%} | {cvar_t_975:.4%}")
117 print(f"{'3. Kernel Density Estimator':<30} |
    ↳ {var_kde_99:.4%} | {cvar_kde_975:.4%}")
118 print("-" * 60 + "\n")
119
120 # Global styling mapping for the comparative charts
121 colors = {'gbm': '#1f77b4', 't': '#ff7f0e', 'kde':
    ↳ '#2ca02c'}
122
123 # =====
124 # PLOT 1: PROBABILITY DENSITY TAIL FOCUS
125 # UPDATED: Plotted 99% VaR and 97.5% CVaR simultaneously
126 # =====
127 plt.figure(figsize=(10, 6))
128 plt.hist(sim_returns_gbm, bins=250, density=True,
    ↳ alpha=0.3, color=colors['gbm'], label='GBM (Normal)')
129 plt.hist(sim_returns_t, bins=250, density=True, alpha=0.3,
    ↳ color=colors['t'], label="Student's t")

```

```

135 plt.hist(sim_returns_kde, bins=250, density=True,
136         ↪ alpha=0.3, color=colors['kde'], label='KDE')
137
138 # Add 99% VaR markers (Dotted lines)
139 plt.axvline(-var_gbm_99, color=colors['gbm'],
140         ↪ linestyle=':', linewidth=2, label=f'GBM 99% VaR
141         ↪ ({var_gbm_99:.2%})')
142 plt.axvline(-var_t_99, color=colors['t'], linestyle=':',
143         ↪ linewidth=2, label=f'Student t 99% VaR
144         ↪ ({var_t_99:.2%})')
145 plt.axvline(-var_kde_99, color=colors['kde'],
146         ↪ linestyle=':', linewidth=2, label=f'KDE 99% VaR
147         ↪ ({var_kde_99:.2%})')
148
149 # Add 97.5% CVaR markers (Solid lines)
150 plt.axvline(-cvar_gbm_975, color=colors['gbm'],
151         ↪ linestyle='-', linewidth=2, label=f'GBM 97.5% CVaR
152         ↪ ({cvar_gbm_975:.2%})')
153 plt.axvline(-cvar_t_975, color=colors['t'], linestyle='-',
154         ↪ linewidth=2, label=f'Student t 97.5% CVaR
155         ↪ ({cvar_t_975:.2%})')
156 plt.axvline(-cvar_kde_975, color=colors['kde'],
157         ↪ linestyle='-', linewidth=2, label=f'KDE 97.5% CVaR
158         ↪ ({cvar_kde_975:.2%})')
159
160 plt.title("Probability Density Focus: 99% VaR vs 97.5%
161         ↪ CVaR Loss Tail", fontsize=13, fontweight='bold')
162 plt.xlabel("Daily Portfolio Returns")
163 plt.ylabel("Density")
164 plt.xlim(np.percentile(sim_returns_t, 0.05),
165         ↪ np.percentile(sim_returns_t, 15))
166 plt.legend(fontsize=9, loc='upper left')
167 plt.grid(True, alpha=0.2)
168 plt.show()
169
170 # =====
171 # PLOT 2: CUMULATIVE DISTRIBUTION FUNCTION (CDF)
172 # =====
173 plt.figure(figsize=(10, 6))
174 for data, name, col in zip([sim_returns_gbm,
175         ↪ sim_returns_t, sim_returns_kde],
176         ↪ ['GBM (Normal)', 'Student's t',
177         ↪ 'KDE'],
178         ↪ [colors['gbm'], colors['t'],
179         ↪ colors['kde']]):
180     sorted_data = np.sort(data)
181     cdf = np.arange(1, len(sorted_data) + 1) /
182         ↪ len(sorted_data)
183     plt.plot(sorted_data, cdf, label=name, color=col,
184             ↪ linewidth=2)
185
186 plt.axhline(1-0.975, color='red', linestyle='--',
187         ↪ linewidth=1.2, label='Significance Level (0.025)')
188 plt.axhline(1-0.95, color='blue', linestyle='--',
189         ↪ linewidth=1.2, label='Significance Level (0.05)')
190 plt.title("Empirical CDF Quantile Trajectory",
191         ↪ fontsize=13, fontweight='bold')
192 plt.xlabel("Daily Portfolio Returns")
193 plt.ylabel("Cumulative Probability")
194 plt.xlim(np.percentile(sim_returns_t, 0.1), 0.01)
195 plt.ylim(0, 0.10)
196 plt.legend(fontsize=10, loc='lower right')
197 plt.grid(True, alpha=0.2)
198 plt.show()
199
200 # =====
201 # PLOT 3: DIRECT METRIC COMPARISON (BAR CHART)
202 # UPDATED: Plotted 99% VaR and 97.5% CVaR on the same
203 ↪ graph
204 # =====
205 fig, ax = plt.subplots(figsize=(10, 6))
206 categories = ['GBM (Normal)', 'Student's t', 'KDE']
207
208 var_values = [var_gbm_99 * 100, var_t_99 * 100, var_kde_99
209         ↪ * 100]
210 cvar_values = [cvar_gbm_975 * 100, cvar_t_975 * 100,
211         ↪ cvar_kde_975 * 100]
212
213 x = np.arange(len(categories))
214 width = 0.35
215
216 rects1 = ax.bar(x - width/2, var_values, width, label='99%
217         ↪ VaR', color='#bcbdd2', alpha=0.85)
218
219 rects2 = ax.bar(x + width/2, cvar_values, width,
220         ↪ label='97.5% CVaR', color='#d62728', alpha=0.85)
221
222 ax.set_title("Direct Value Comparison: 99% VaR vs 97.5%
223         ↪ CVaR (%)", fontsize=13, fontweight='bold')
224 ax.set_xticks(x)
225 ax.set_xticklabels(categories)
226 ax.set_ylabel("Loss Scale (%)")
227 ax.legend(fontsize=10, loc='upper left')
228 ax.grid(True, axis='y', alpha=0.3)
229
230 def autolabel(rects):
231     for rect in rects:
232         height = rect.get_height()
233         ax.annotate(f'{height:.2f}%',
234                 ↪ xy=(rect.get_x() + rect.get_width() /
235                 ↪ 2, height),
236                 ↪ xytext=(0, 3),
237                 ↪ textcoords="offset points",
238                 ↪ ha='center', va='bottom', fontsize=9,
239                 ↪ fontweight='bold')
240
241 autolabel(rects1)
242 autolabel(rects2)
243 plt.tight_layout()
244 plt.show()
245
246 # =====
247 # PLOT 4: GBM PRICE TRAJECTORIES
248 # =====
249 plt.figure(figsize=(10, 5))
250 plt.plot(price_paths_gbm, linewidth=1.5,
251         ↪ color=colors['gbm'], alpha=0.6)
252 plt.title("Geometric Brownian Motion (Normal) Simulated
253         ↪ Price Paths", fontsize=12, fontweight='bold')
254 plt.xlabel("Time Horizon (Days)")
255 plt.ylabel("Asset Price")
256 plt.grid(True, alpha=0.3)
257 plt.axhline(initial_price, color='black', linestyle='--',
258         ↪ alpha=0.5)
259 plt.show()
260
261 # =====
262 # PLOT 5: STUDENT'S T PRICE TRAJECTORIES
263 # =====
264 plt.figure(figsize=(10, 5))
265 plt.plot(price_paths_t, linewidth=1.5, color=colors['t'],
266         ↪ alpha=0.6)
267 plt.title("Student's t-Distribution Simulated Price
268         ↪ Paths", fontsize=12, fontweight='bold')
269 plt.xlabel("Time Horizon (Days)")
270 plt.ylabel("Asset Price")
271 plt.grid(True, alpha=0.3)
272 plt.axhline(initial_price, color='black', linestyle='--',
273         ↪ alpha=0.5)
274 plt.show()
275
276 # =====
277 # PLOT 6: KDE PRICE TRAJECTORIES
278 # =====
279 plt.figure(figsize=(10, 5))
280 plt.plot(price_paths_kde, linewidth=1.5,
281         ↪ color=colors['kde'], alpha=0.6)
282 plt.title("Kernel Density Estimator (KDE) Simulated Price
283         ↪ Paths", fontsize=12, fontweight='bold')
284 plt.xlabel("Time Horizon (Days)")
285 plt.ylabel("Asset Price")
286 plt.grid(True, alpha=0.3)
287 plt.axhline(initial_price, color='black', linestyle='--',
288         ↪ alpha=0.5)
289 plt.show()
290
291 # =====
292 # PREPARATION FOR DISTRIBUTION OVERLAYS (PLOTS 7-9)
293 # =====
294 x_axis = np.linspace(np.min(sim_returns_t),
295         ↪ np.max(sim_returns_t), 1000)
296 normal_curve = stats.norm.pdf(x_axis, loc=mu_norm,
297         ↪ scale=sigma_norm)
298

```

```

263 # =====
264 # PLOT 7: HISTOGRAM - GEOMETRIC BROWNIAN MOTION
265 # =====
266 plt.figure(figsize=(10, 5))
267 plt.hist(sim_returns_gbm, bins=150, density=True,
268         ↪ alpha=0.6, color=colors['gbm'], edgecolor='black',
269         ↪ linewidth=0.5)
270 plt.plot(x_axis, normal_curve, color='red', linestyle='-',
271         ↪ linewidth=2, label='Theoretical Normal Curve')
272 plt.title("Distribution of Returns - Geometric Brownian
273         ↪ Motion (Normal)", fontsize=12, fontweight='bold')
274 plt.xlabel("Simulated Daily Returns")
275 plt.ylabel("Frequency Density")
276 plt.xlim(np.percentile(sim_returns_t, 0.1),
277         ↪ np.percentile(sim_returns_t, 99.9))
278 plt.legend()
279 plt.grid(True, alpha=0.2)
280 plt.show()
281
282 # =====
283 # PLOT 8: HISTOGRAM - STUDENT'S T-DISTRIBUTION
284 # =====
285 plt.figure(figsize=(10, 5))
286 plt.hist(sim_returns_t, bins=150, density=True, alpha=0.6,
287         ↪ color=colors['t'], edgecolor='black', linewidth=0.5)
288 plt.plot(x_axis, normal_curve, color='red', linestyle='-',
289         ↪ linewidth=2, label='Theoretical Normal Curve')
290 t_curve = stats.t.pdf(x_axis, df=df_est, loc=loc_est,
291         ↪ scale=scale_est)
292 plt.plot(x_axis, t_curve, color='black', linestyle='--',
293         ↪ linewidth=2, label='Fitted Student's t Curve')
294 plt.title("Distribution of Returns - Student's
295         ↪ t-Distribution", fontsize=12, fontweight='bold')
296 plt.xlabel("Simulated Daily Returns")
297 plt.ylabel("Frequency Density")
298 plt.xlim(np.percentile(sim_returns_t, 0.1),
299         ↪ np.percentile(sim_returns_t, 99.9))
300 plt.legend()
301 plt.grid(True, alpha=0.2)
302 plt.show()
303
304 # =====
305 # PLOT 9: HISTOGRAM - KERNEL DENSITY ESTIMATOR (KDE)
306 # =====
307 plt.figure(figsize=(10, 5))
308 plt.hist(sim_returns_kde, bins=150, density=True,
309         ↪ alpha=0.6, color=colors['kde'], edgecolor='black',
310         ↪ linewidth=0.5)
311 plt.plot(x_axis, normal_curve, color='red', linestyle='-',
312         ↪ linewidth=2, label='Theoretical Normal Curve')
313 plt.plot(x_axis, kde.pdf(x_axis), color='darkgreen',
314         ↪ linestyle='-', linewidth=2, label='KDE Continuous Fit
315         ↪ Line')
316 plt.title("Distribution of Returns - Kernel Density
317         ↪ Estimator (KDE)", fontsize=12, fontweight='bold')
318 plt.xlabel("Simulated Daily Returns")
319 plt.ylabel("Frequency Density")
320 plt.xlim(np.percentile(sim_returns_t, 0.1),
321         ↪ np.percentile(sim_returns_t, 99.9))
322 plt.legend()
323 plt.grid(True, alpha=0.2)
324 plt.show()

```